

301.1E

DEFINE · AUDIT · PLAN · AI-ASSISTED SOFTWARE ENGINEERING

Turn other people's intent into a plan you can defend.

A practical lab for engineers who receive a PRD, an architecture and a design from other functions, and own the tech spec and the plan that agents will build from.

WHAT PARTICIPANTS LEARN

Move from reading specs to commissioning work.

- Read product, architecture and design against each other and against the code.
- Send each gap to the person who owns it, instead of deciding it.
- Steer an agent to a tech spec that names what is still open.
- Know what audit and plan run underneath, what is a model's judgment and what is a deterministic check.

FORMAT

Seven stages · 120 minutes

- Hands-on with SPEED throughout, including its internals
- Engineers play every role: PM, designer, architect and engineer
- One brownfield card-payments service and one feature
- Ends with a verified task plan ready for Execute

THE CONCRETE EXAMPLE THROUGHOUT

A traveller in Tokyo is refused a 2,499 pound television by a rule written at launch, while forty stolen cards are approved across twenty merchants in three minutes. Product, architecture and design have each written their part of the fix. Engineering owns the rest.

`Orient` → Define → Draft tech spec → Audit → Plan → Verify → Handoff
↖ a gap the audit finds goes back to its owner

The promise to a client

Participants leave able to take a feature from other functions' documents to a verified plan, knowing which parts a person decided, which parts SPEED decided, and which questions are still owned by someone else.

INPUTS, OUTPUTS AND CONTINUITY

Every stage produces something the next stage can use.

The provided documents stay with their owners. Engineering writes one thing, the tech spec, and everything after it is checked against all of them.

STAGE	COMMAND AND PURPOSE	INPUTS	PARTICIPANT OUTPUT	HANDOFF
01	Orient <code>npm test</code>	Problem statement, existing gateway, risk rules and tests	Baseline test run	The behaviour the specs must change, pinned as tests.
02	Define <code>/define/adaptive-auth</code>	Two PRDs, platform and feature architecture, two designs, product vision	Suggestions, each accepted or dismissed by its owner	Decisions the tech spec can rely on. Open questions keep a named owner.
03	Draft the tech spec agent, engineer-steered	Upstream documents, Define decisions, codebase	specs/tech/adaptive-auth/	A draft that names what is undecided.
04	Audit <code>speed audit --feature</code>	Every spec in the feature	Audit report, sizing, split	Split tech files go to Plan. Upstream gaps go back to Define.
05	Plan <code>speed plan --feature</code>	Audited specs, product vision, codebase	Task records, dependency graph, gate results	A plan written by the Architect and amended by SPEED.
06	Verify <code>speed verify</code>	Task plan, specs	Verify report, repaired tasks	A plan checked by something other than its author.
07	Handoff to Execute	Verified plan, tech files, open decisions	Execution brief	No undecided threshold or policy travels as a task.

Current delivery note. Architecture specs, multi-file features, `audit --feature`, feature-level `plan` and role switching in Define are being added to SPEED alongside this course. The fixture and stage guides are written against that interface.

START WITH WHAT EXISTS

Read the system, then the documents, then write.

In the first half the engineer does most of the work and SPEED assists. Each stage separates what the engineer decides from what SPEED surfaces and runs.

NPM TEST

01 Orient

What does the system do today, before anyone's document describes what it should do?

ENGINEER DECIDES What the system does today	ENGINEER RUNS npm test, then reads the gateway and risk rules	SPEED SURFACES Nothing yet	SPEED RUNS Nothing yet
ADAPTIVE AUTHORISATION EXAMPLE The traveller declined, a decline that tells the merchant nothing, the card-testing burst approved in full, and velocity counted separately on every replica.		PRODUCES Baseline test run The behaviour the specs must change.	

SPEED DASHBOARD · DEFINE

02 Define

What do the provided documents leave undecided, and who has to decide it?

ENGINEER DECIDES What is a gap and whose it is; then, as the owner, accept or dismiss with a reason	ENGINEER RUNS Section suggestions, role switch, commit	SPEED SURFACES Live template and structure checks det ; cross-spec, codebase, sizing, vision LLM det ; ambiguity detector LLM ; blast radius det	SPEED RUNS Commitment gate det ; auto-ratifies when nobody else is eligible det
ADAPTIVE AUTHORISATION EXAMPLE Asks the checkout PM for a baseline behind "30% fewer false declines". Tells the architect a stateless scoring service cannot see an attack spread across twenty merchants. As the PM, records that risk appetite waits on Risk.		PRODUCES Owner decisions on suggestions A ratification record exists; independent approval may not.	

AGENT-DRAFTED · ENGINEER-STEERED

03 Draft the tech spec

What did the agent decide that nobody asked it to?

ENGINEER DECIDES What the agent assumed; what stays in Unresolved Questions	ENGINEER RUNS An agent drafting the tech spec from the documents and code	SPEED SURFACES Fix suggestions LLM ; codebase references checked as you type det	SPEED RUNS Nothing until audit
ADAPTIVE AUTHORISATION EXAMPLE Removes a fallback the agent invented for when the bank does not answer. Keeps the risk narrative out of the authorisation path until the architect decides.		PRODUCES specs/tech/adaptive-auth/ A draft ready to audit.	

What this changes in practice

A gap in someone else's document stops being a guess an engineer makes quietly. It becomes a suggestion with an owner and a recorded answer.

LET THE TOOLS PUSH BACK

Audit the feature, then read what one command runs.

From here SPEED does most of the work. The engineer's job becomes judging what came back and knowing which results came from a model.

SPEED AUDIT --FEATURE

04 Audit

Is the feature ready to plan, and what can the audit not see?

ENGINEER DECIDES	ENGINEER RUNS	SPEED SURFACES	SPEED RUNS
Whether a finding is real, whose it is, whether to split	<code>speed audit --feature adaptive-auth</code>	Findings by level, sizing estimate, split recommendation	Spec type from path, templates and linked specs loaded <code>det</code> ; audit agent, four levels, read-only <code>LLM</code>

ADAPTIVE AUTHORISATION EXAMPLE

Flags a design TODO, a success criterion with no number, and a held-payment flow with no screen. Does not notice the architecture cannot detect card testing: it reads documents, not code.

PRODUCES

Audit report + split tech files

Passing the audit does not mean the requirements are right.

SPEED PLAN --FEATURE

05 Plan

Does the task graph still say what the specs meant?

ENGINEER DECIDES	ENGINEER RUNS	SPEED SURFACES	SPEED RUNS
Whether to override a gate; whether the graph makes sense	<code>speed plan --feature adaptive-auth</code> , one command	Guardian, audit, spec alignment, coverage and decomposition reports	Seven steps before tasks are written:

1	<code>LLM</code>	Guardian checks specs against the product vision and anti-goals	Can stop the plan
2	<code>LLM</code>	Audit runs again on every spec, as a gate	Can stop the plan
3	<code>det</code>	Code index: project map, semantic graph, file skeletons, spec-to-code alignment	Falls back with a warning
4	<code>det</code>	Related-spec scoring picks what reaches the Architect, within a token budget	
5	<code>LLM</code>	Architect writes tasks and a data contract, in phases above ten estimated tasks	
6	<code>LLM</code>	Coverage verifier adds tasks for requirements the Architect missed	Changes the plan
7	<code>det</code>	Decomposition gate: size, cluster coordination, bridge symbols, file ownership	Auto-patches, then can stop

ADAPTIVE AUTHORISATION EXAMPLE

Both product specs change the gateway, so file ownership forces an order. The size check passes every task because the files are small: a green check only as good as its measure.

PRODUCES

Task records + dependency graph

A plan more than one actor wrote.

CHECK, THEN HAND OVER

Verify the plan, and send only what has been decided.

Verification is the last place a person can catch a plan that drifted from the specs before agents start building.

SPEED VERIFY

06 Verify

Which verifier findings need an owner, and which are mechanical?

ENGINEER DECIDES Which questions go to an owner; which fixes to accept	ENGINEER RUNS speed verify	SPEED SURFACES Verifier findings, human questions separated from mechanical fixes	SPEED RUNS Spec-to-task traceability det ; blind Plan Verifier LLM ; up to three repair cycles that edit task records LLM
--	--	---	---

ADAPTIVE AUTHORISATION EXAMPLE

Traceability finds no task for merchant notification before the model runs. The verifier asks what happens when the bank does not answer, which only Risk and product can decide.

PRODUCES

Verify report
"Blind" means blind to the Architect's reasoning, not to structure.

TO EXECUTE

07 Handoff

What goes to Execute, and what must not?

ENGINEER DECIDES What is decided enough to build	ENGINEER RUNS Assembles the brief	SPEED SURFACES The verified plan and its open questions	SPEED RUNS Nothing
--	---	---	------------------------------

ADAPTIVE AUTHORISATION EXAMPLE

Decision thresholds, decline reason wording and block length stay with Risk, Legal and product. None of them reaches an agent as a task.

PRODUCES

Execution brief
Ready for the Execute course.

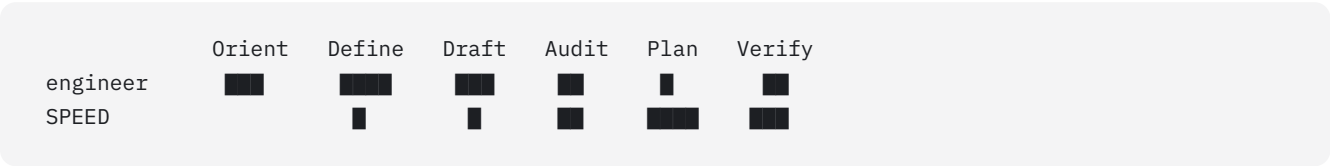
Human ownership stays visible

Engineering can ask "what fraud rate will we accept for fewer false declines?" Only Risk can answer it, and the plan waits for the answer rather than inventing one.

WHO DOES THE WORK

The work moves from the engineer to SPEED.

Engineers learn where SPEED grounds itself, which results are a model's judgment, and where it changes their artifacts without asking.



What SPEED grounds itself on

Semantic graph	Symbols, references, domain clusters and blast radius across the codebase det
File skeletons	Signatures without bodies, about ten to one. What the Architect actually sees of existing code det
Spec-to-code alignment	Every code reference in a spec marked confirmed, missing or divergent det
Spec traceability	Every requirement matched to a task before the verifier's model runs det

Where SPEED acts without asking

Define	Ratifies on the author's behalf when nobody else is eligible
Plan	Adds tasks for missed requirements; patches dependencies and re-runs the decomposition gate; reuses cached results while specs are unchanged
Verify	Edits task records in up to three repair cycles

Task size, measured three ways

Audit	About one developer-day per task; over ten tasks phases the Architect LLM
Architect	Told to write 15 to 30 minute tasks; not enforced LLM
Decomposition gate	Lines in the files a task touches; warns above 3,000 det

ONE FEATURE, FIVE LENSES

A real feature touches every function.

Adaptive authorisation was chosen because no single document can be right on its own. Each lens below has an owner, and the engineer meets all of them.

LENS	WHAT THE ENGINEER MEETS	OWNER
Real-time fraud detection	A 40 ms risk budget, six replicas with no shared state, and a model that emits a single score	Architect, data science
GenAI risk	Attackers who script card testing across merchants, and our own LLM narrative fed by merchant-controlled text	Architect, risk platform PM
Customer experience	Asking before refusing, and what a cardholder sees while a payment is held	Checkout PM, designer
Legal and risk	Explaining automated declines, 3-D Secure exemptions, and the fraud rate we will accept	Legal, Risk
Business value	Lost high-value sales against fraud losses and scheme fees on every test	Product

WHAT THE CLIENT RECEIVES

- A shared language for readiness, ownership and open questions across product, design, architecture and engineering.
- Seven stage guides grounded in one brownfield payments feature.
- A visible path from three functions' documents to a verified task plan.
- Practice telling a model's judgment from a deterministic check, and both from what only a person can see.
- Engineers who understand what other functions commit to.

WHAT IS NOT PROMISED

- No audit or plan result is presented as proof the requirements are right.
- The lab does not set fraud thresholds, risk appetite or legal policy.
- The lab does not replace the client's own product, design or architecture owners.
- Execution and approval to ship belong to the courses that follow.

Resulting behaviour

Before commissioning agent work, participants can say what each document decided, what it left open, who owns each open question, what the tools checked, and why the plan says what it says.